

```
Directory: /home/guest/uart2spi/trunk/rtl/msg_hand
verilog, 1 file(s)
```

FILENAME	MODULE	IO	NET	MCCABE	TIME
uart_msg_handler.v		0	0	1	0.0192
uart_m_ndler		0	0	1	

The default behaviour is to dump the output to the terminal. It can be redirected to a file with the “-output-file=FILE” option. You can also specify an output file format, such as “csv” with the “-output-format=FORMAT” option:

```
$ hct --output-file=/home/guest/project-metrics.csv
--output-format=csv rtl/spi
$ cat /home/guest/project-metrics.csv
```

```
Directory: /home/guest/uart2spi/trunk/rtl/spi
verilog, 4 file(s)
FILENAME, MODULE, IO, NET, MCCABE, SLOC, COMMENT_LINES, TIME
spi_ctl.v, , 20, 1, 1, 110, 48, 0.1644
, spi_ctl, 20, 1, 1, 68, 6,
spi_core.v, , 0, 0, 1, 46, 43, 0.0073
, spi_core, 0, 0, 1, 4, 1,
spi_cfg.v, , 0, 0, 1, 46, 43, 0.0075
, spi_cfg, 0, 0, 1, 4, 1,
spi_if.v, , 15, 3, 1, 80, 44, 0.0948
, spi_if, 15, 3, 1, 38, 2,
```

There are various yyparse options that are helpful to understand the lexical parsing of the source code. They can be invoked using following command:

```
$ hct --yydebug=NN sources
```

The NN options and their meaning is listed below:

- 0x01 Lexical tokens
- 0x02 Information on States
- 0x04 Shift, reduce, accept driver actions
- 0x08 Dump of the parse stack
- 0x16 Tracing for error recovery
- 0x31 Complete output for debugging

HCT can also be used with VHDL and Cyclicity CDL (cycle description language) programs. For VHDL, the filenames must end with a .vhd extension. You can rename .vhd files recursively in a directory (in bash, for example) using the following script:

```
for file in `find $1 -name "*.vhd"`
do
mv $file $(file/.vhd/.vhd1)
done
```

The “\$1” refers to the project source directory that is passed as an argument to the script.

Let us take example of sha256core written in VHDL, which is also included in this month’s EFY DVD. The execution of HCT on the sha256core project is as follows:

```
$ hct rtl
Directory: /home/guest/sha256core/trunk/rtl
vhd1, 6 file(s)
```

FILENAME	MODULE	IO	NET	MCCABE	TIME
sha_256.vhd1		29	0	1	0.9847
sha_256		29	0	1	
sha_fun.vhd1		1	1	1	0.3422
		1	1	1	
msg_comp.vhd1		20	0	1	0.4169
msg_comp		20	0	1	
dual_mem.vhd1		7	0	3	0.0832
dual_mem		7	0	3	
ff_bank.vhd1		3	0	2	0.0260
ff_bank		3	0	2	
sh_reg.vhd1		19	0	1	0.6189
sh_reg		19	0	1	

The “-T” option enables the use of threads to speed up computation. The LZRW1 (Lempel-Ziv Ross Williams) compressor core project implements a lossless data compression algorithm. The output of HCT on this project, without threading and with threads enabled, is shown below:

```
$ time hct HDL
```

```
Directory: /home/guest/lzrw1-compressor-core/trunk/hw/HDL
vhd1, 8 file(s)
...
real 0m3.725s
user 0m3.612s
sys 0m0.013s
```

```
$ time hct HDL -T
```

```
Directory: /home/guest/lzrw1-compressor-core/trunk/hw/HDL
vhd1, 8 file(s)
...
real 0m2.301s
user 0m7.029s
sys 0m0.051s
```

The supported input options for HCT can be viewed with the “-h” command.

The invocation of HCT can be automated, rechecked for each code

check-in that happens to a project repository. The complexity measure is thus recorded periodically. The project team will then be able to monitor, analyse the complexity of each module and decide on any code refactoring strategies. ●

The author is a software enthusiast

EFY Note

The source code of this project is included in this month’s EFY DVD and is also available for free download at source.efymag.com